

CSPLib prob018: Water Bucket Problem with Scheduling Extension

This project addresses the classic water bucket pouring puzzle from CSPLib problem 018. The base problem is the standard 8-5-3 bucket puzzle: given three containers with fixed capacities and an initial state, determine a sequence of pours that reaches a target distribution (for example, 4 litres in each of the two larger buckets). The project extends this idea beyond the fixed textbook version.

The implemented extension generalises the problem to an arbitrary number of buckets ($n \geq 2$), arbitrary capacities, arbitrary initial states, and partial goal definitions. More importantly, the final objective is not only to reach the target amounts, but to do so by minimising a scheduling objective: total makespan and number of operations. In other words, the system models an operational planning problem in which each transfer action is scheduled over time, rather than only as a graph-search transition.

The project is implemented in Python with a MiniZinc constraint-programming model and uses Gecode as the solver. The key files are:

- `files/proj.py` — entry point that builds the instance and solves it
- `files/water_bucket.mzn` — generalised MiniZinc model
- `files/water_bucket_scheduling.mzn` — scheduling-oriented extension
- `files/instances/classic_8_5_3.json` — standard benchmark
- `files/instances/four_bucket_extension.json` — extended multi-bucket instance

Video submission:

Python 3.10+
MiniZinc 2.10.1 or newer

Commands:

```
git clone https://github.com/Guilherme-Corvelo/IAES-2627-proj.git cd IAES-2627-proj p
ython -m venv .venv source .venv/bin/activate pip install -r files/requirements.txt
```

Then run:

```
cd files python proj.py --file instances/classic_8_5_3.json python proj.py --file ins
tances/four_bucket_extension.json
```

The first example solves the classic 8-5-3 problem, while the second demonstrates the extended multi-bucket version. The goal values can also be specified directly on the command line:

```
python proj.py --capacities 8 5 3 --initial 8 0 0 --goal 4 4 -1
```

Reflection on external resources and AI use

The project made use of generative AI assistance to help translate the mathematical idea into Python and MiniZinc code, especially validation logic. However, all generated code was reviewed, adjusted, and tested manually against the repository's own reference solver and benchmark instances.

